

Efficient Large Language Model Acceleration Using HBM-Based PIM Architectures

MCCSys 6th workshop

Taewoon Kang, Namhun Kim, Jihun Lee, Murali Annavaram, Gunjae Koo

2026-06-29



KOREA
UNIVERSITY



USC University of
Southern California

LLMs Are Ubiquitous

Large Language Models are now deployed across virtually every domain of daily life and industry.

AI

Conversational AI

ChatGPT, Claude, Gemini — billions of queries daily across consumer and enterprise settings.

DEV

Software Engineering

Code generation, review, and documentation — deeply integrated via GitHub Copilot, Cursor.

EDU

Education

Personalized tutoring, language learning, essay feedback, and adaptive curricula at scale.

SCI

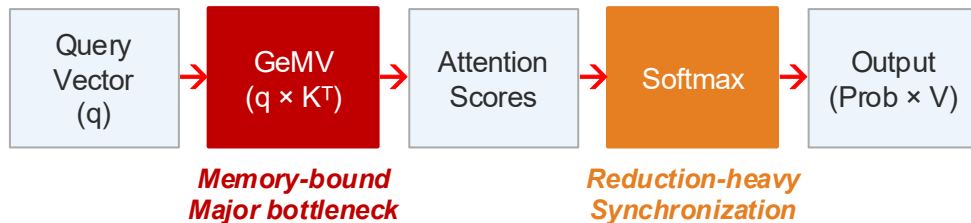
Scientific Research

Literature synthesis, hypothesis generation, and accelerating experimental design.

LLM computation pipeline

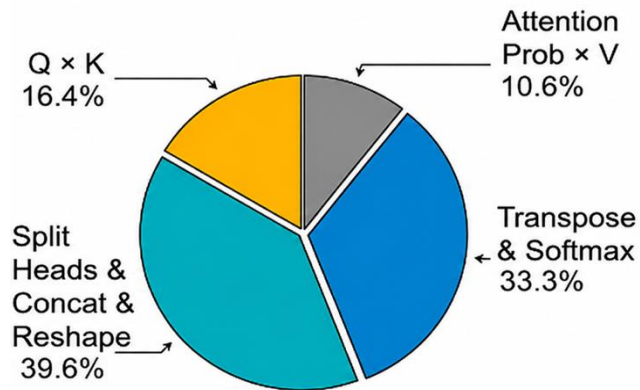
Autoregressive Decoding: Attention Pipeline

During LLM inference, each token generation step executes the following pipeline — dominated by two operations:



Key Insight: GeMV and Softmax together dominate attention latency.

Attention Latency Breakdown



Transpose & Softmax alone account for 33.3% of attention latency. MatMul (GeMV) accounts for 27%.

Where is the Bottleneck?

GeMV's Memory Troubles

Why GeMV is a Memory Bottleneck

GeMV ($q \times K^T$) accesses a large key matrix stored in DRAM for every single token — making it heavily memory-bound. The arithmetic intensity is too low to benefit from compute-rich GPUs.

Accelerate GeMV using PIM

Why GeMV is a Memory Bottleneck

GeMV ($q \times K^T$) accesses a large key matrix stored in DRAM for every single token — making it heavily memory-bound. The arithmetic intensity is too low to benefit from compute-rich GPUs.

Processing-in-Memory (PIM): The Solution

Samsung HBM-PIM places SIMD compute units directly alongside DRAM banks, enabling matrix-vector operations without transferring data to the host. Each near-bank SIMD unit processes data as it is read from the row buffer.

Bank-Level Parallelism

- All-bank PIM mode executes the same PIM operation across 16 banks in parallel, improving GeMV throughput.

FP16 SIMD Units

- Each bank pair has FP16 SIMD multiply/add units that process 16 FP16 elements.

What about Softmax?

Why not Softmax in-memory?

Even with PIM accelerating GeMV, Softmax still runs on the host — incurring data movement overhead. Why is Softmax hard to compute in memory?

1 e^x is Expensive in Hardware

- The exponential function e^x is non-linear and has no native hardware support.
- Implementing it directly requires a dedicated high-precision unit — costly in area and power.
- Near-bank PIM units are intentionally lightweight: they contain only simple FP16 MULT/ADD units.
- A full e^x unit would far exceed the power budget of the near-bank processing unit.

2 Softmax Requires the Full Vector

- Standard Softmax: $y_i = \exp(a_i - a_{\max}) / \sum \exp(a_j - a_{\max})$
- Computing $a_{\max}$ requires a global max-reduction across all N vector elements.
- Computing the denominator Σ requires another global sum-reduction across all elements.
- Both reductions require synchronization — fundamentally incompatible with bank-level parallel PIM execution.

Two Ideas

- Make Softmax hardware friendly
- Eliminate exponentiation need

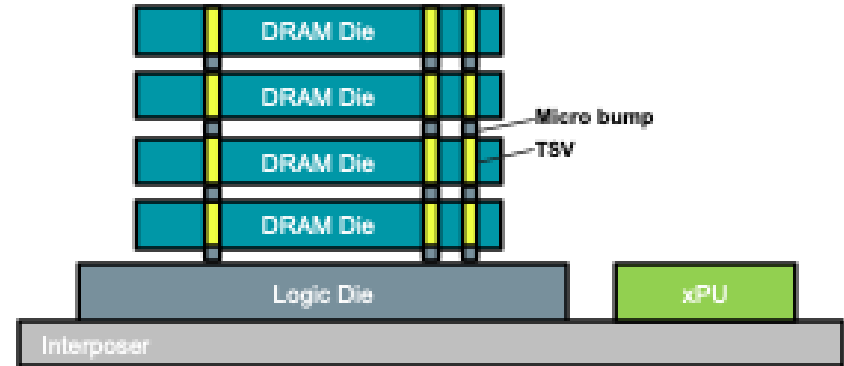
Outline

- **HBM & HBM-PIM Background**
- **Proposed architecture**
- **Execution flow**
- **Evaluation methodology**
- **Evaluation result**

Overview of HBM

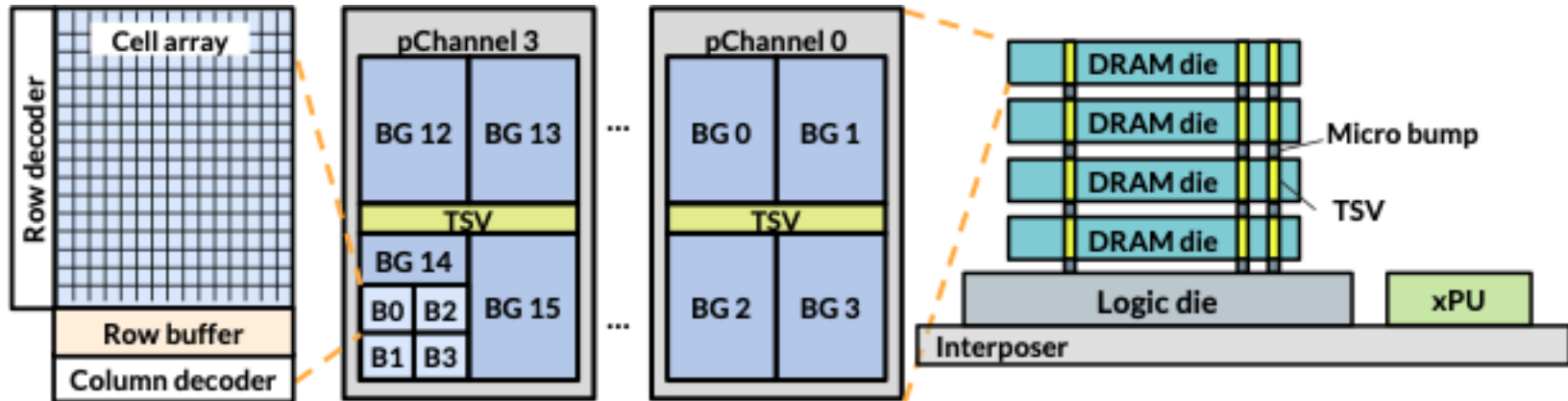
- High Bandwidth Memory

- Vertically stacked memory chips
- Wire → TSV & Micro bump
- Connect between HBM and processor through Interposer
- Logic die: consist of I/O buffer and essential test logic



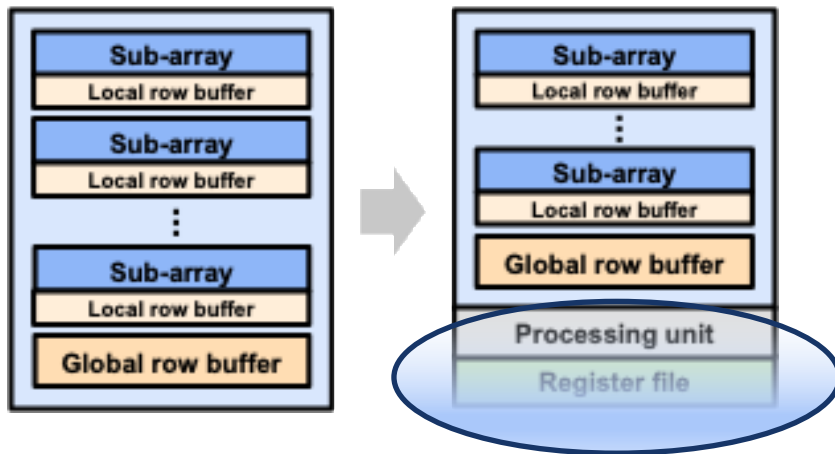
Overview of Basic HBM

- High Bandwidth Memory
 - DRAM die
 - Logic die
 - Channel, Pseudo channel, Bank group, Bank, Cell array, Row buffer,



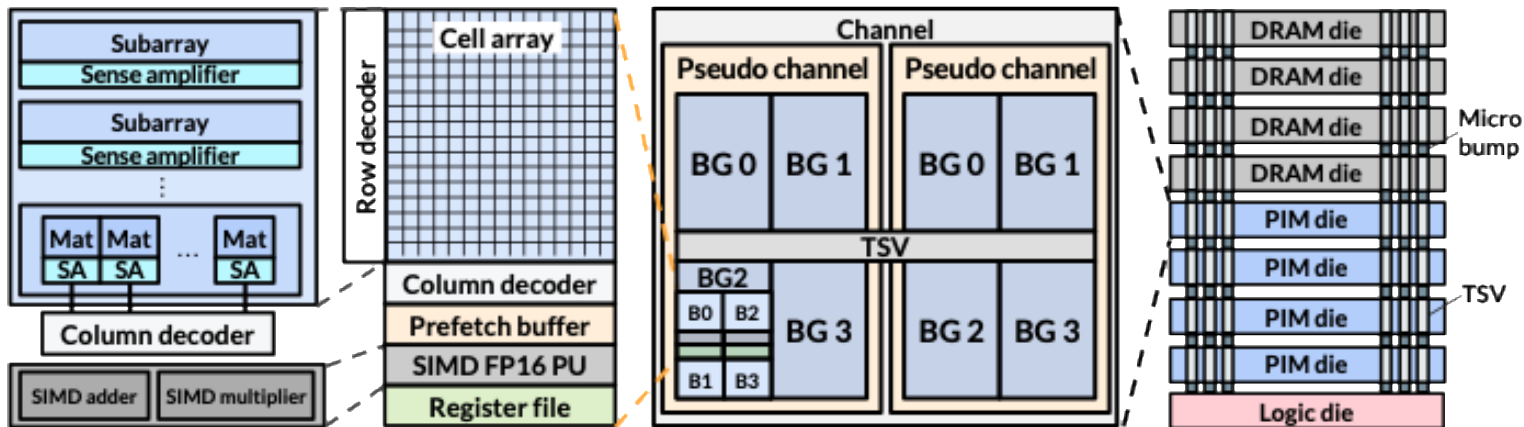
HBM+PIM overview

- Samsung HBM-PIM = FIMDRAM = Aquabolt-XL
 - Modify some HBM2 DRAM layer to embed SIMD FPU
 - For **GeMV** acceleration
 - Reduce the number of sub-arrays by half → execution units implemented (Cell array is composed with multiple sub-arrays)



HBM-PIM overview

- Samsung HBM-PIM = FIMDRAM = Aquabolt-XL
 - PIM units respond to standard DRAM commands(RD or WR)
 - Operate with unmodified JEDEC standard memory controller
 - A PIM unit can get 16 x 16-bit(32B) operands from bank, register
 - Unlike standard DRAM devices, all banks can be accessed concurrently (All bank(AB)-mode)



Idea 1: Replace Softmax to ConSmax

Standard Softmax

Given input scores (after GeMV) $a_1, a_2, \dots, a_n$:

- ① Find maximum: $a_{\max} = \max(a_i)$
- ② Compute $\exp(a_i - a_{\max})$ for each i
- ③ Sum all exponents (global reduction)
- ④ Divide each by the sum $\rightarrow y_i$

$$y_k = \frac{\exp(a_k - a_{\max})}{\sum_{i=1}^n \exp(a_i - a_{\max})}$$



ConSmax (Constant Softmax)

Replace reductions with learned constants:

$\beta \approx \max(a_i)$ (pre-trained parameter)
 $\gamma \approx \sum \exp(a_i - \max)$ (pre-trained parameter)

Then softmax becomes:

$$y_i = \exp(a_i - \beta) / \gamma$$

$$\text{ConSmax}(x_i) = \frac{\exp(x_i - \beta)}{\gamma}$$

⚠ Steps ① and ③ require global synchronization across all elements — bottleneck in parallel hardware

✓ Each element computed independently
No global synchronization required $\rightarrow$ PIM-friendly!

Idea2: Avoiding expensive exponential operation

⚠ **Problem:** Hardware cannot efficiently compute e^x directly.

The exponential function is non-linear with no native hardware support in lightweight PIM units.

Solution: Convert to Base-2

Instead of computing e^x , it as an equivalent base-2 expression that uses only multiply and bit-shift — operations PIM hardware already supports.

$$e^x = 2^{x \log_2 e}$$

✓ e^x computed via multiply + bit-shift + small LUT — no dedicated exponential hardware required.

Avoiding Direct Exponential Computation

The Problem

- Computing e^x directly in hardware is costly, since the exponential function is non-linear and has no native hardware support — making it impractical to implement efficiently near the memory banks.

The Solution: Base-2 Reformulation

- SoftmaxPIM avoids this by rewriting the exponential using the identity below, transforming it into an operation that hardware already handles efficiently: a bit shift combined with a small lookup table.

$$e^x = 2^{x \log_2 e}$$

1 Multiply

x is multiplied by the preloaded constant $\log_2 e$, producing $y = x \cdot \log_2 e$ in fixed-point form.

2 Decompose

y is split into an integer part I and a fractional part F , so that $y = I + F$.

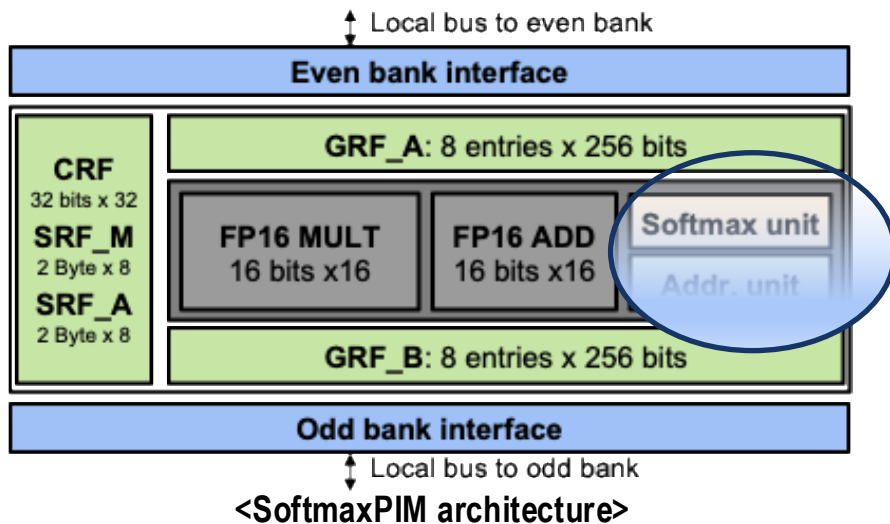
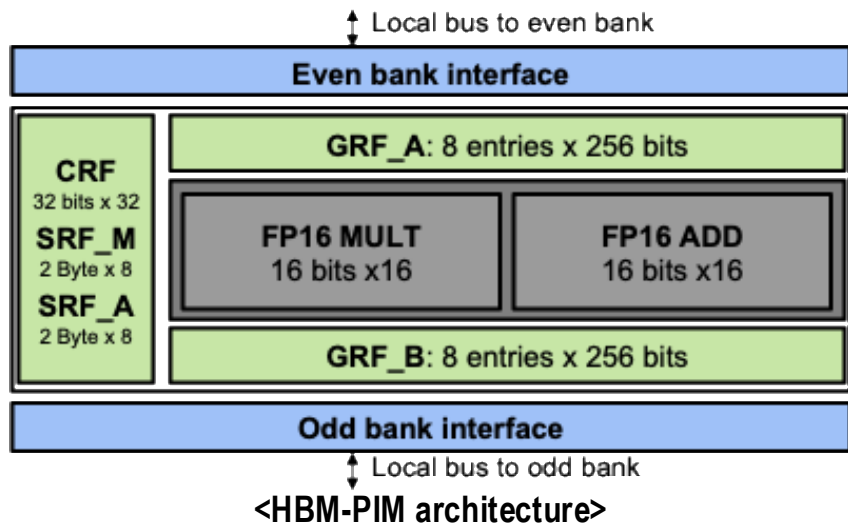
3 Shift + LUT

2^I is computed via a simple bit shift; 2^F is fetched from a small lookup table — then multiplied together.

Proposed architecture

Key Addition: Softmax Unit + Address Generator

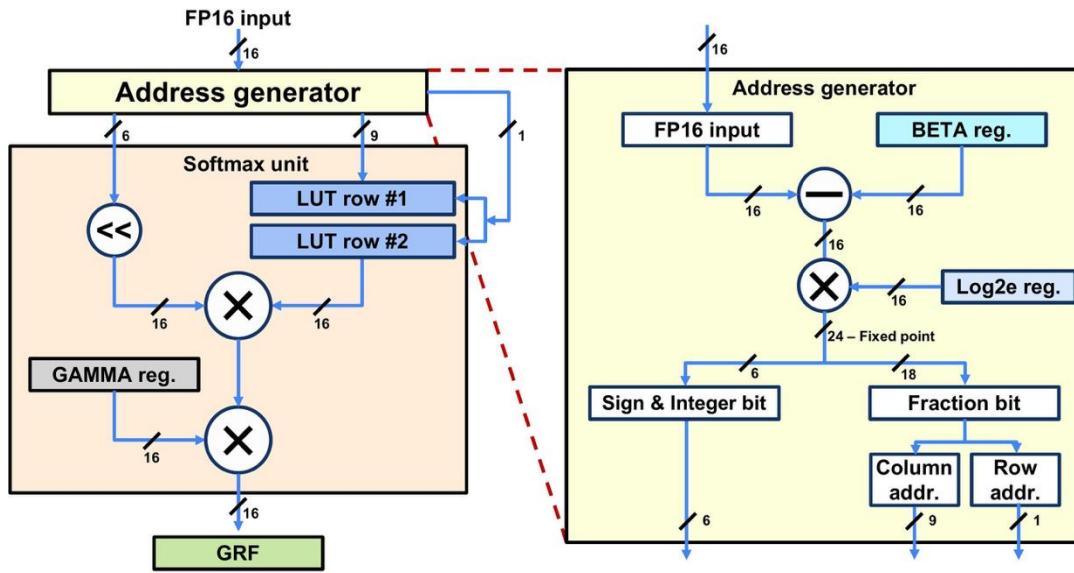
- Built on the baseline HBM-PIM architecture, SoftmaxPIM adds a lightweight Softmax unit and an Address generator alongside the existing FP16 MULT/ADD units within each bank interface — enabling softmax to be computed entirely in memory, immediately after GeMV, with no host transfer required.



Proposed architecture

Softmax unit:

- Instead of computing the exact softmax with expensive global reductions, SoftmaxPIM employs ConSmax — an approximation that replaces the per-vector maximum with a learned constant β , and the global summation with a pre-trained constant γ , enabling fully parallel, independent per-element execution.



Execution flow

① GeMV Operation

- Host writes GeMV command sequence to CRF; places input vector in GRF
- HBM transitions to All-Bank PIM (AB-PIM) mode
- Each column command triggers SIMD vector–vector multiply across 16 lanes
- Partial results accumulated locally in GRF across all banks
- MAC operations complete — dot-product results remain in GRF



② Softmax Operation

MOV

MOV transfers β (BETA) and γ (GAMMA) to softmax unit registers

AGEN

AGEN generates LUT addresses from scaled input $x_i = S_i - \beta$

LDL

LDL fetches precomputed 2^F values from reserved DRAM LUT rows

MERGE

MERGE combines 2^I (shift) and 2^F (LUT) to reconstruct exponential

SCALE

SCALE applies $1/\gamma$ normalization — result written back to GRF

Evaluation methodology

Simulator

DRAMSim3-based cycle-accurate in-house simulator.
Evaluates a fused kernel: GeMV ($N \times N$) immediately followed by softmax on the resulting $N \times 1$ vector.

Matrix Sizes

N swept from 2,048 to 16,384 in powers of two — covering Llama 3 8B (4,096), 70B (8,192), and 405B (16,384).

HBM Configuration

Evaluated on both HBM2 (JEDEC HBM2 spec) and HBM3 (JEDEC HBM3 spec), each with 4 of 8 DRAM dies configured as PIM dies.

Baselines

Apple M4 CPU (2.9 TFLOPS FP16), NVIDIA RTX 3090 GPU (29.8 TFLOPS FP16), and AttAcc — a prior PIM-based in-memory attention accelerator.

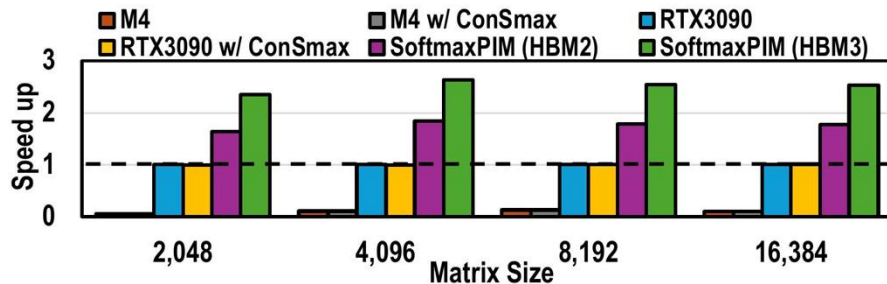
Area & Power

Hardware synthesized using Synopsys Design Compiler with a 14 nm SAED FinFET library, then scaled to 130 nm via DeepScaleTool.

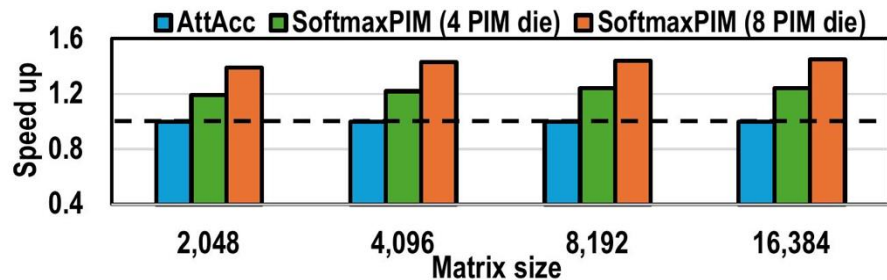
Model Accuracy

A nanoGPT model trained for 30,000 iterations is evaluated on WikiText-103, measuring perplexity and token accuracy.

Evaluation: Performance



<Normalized speedup vs RTX 3090>



<GeMV-softmax speedup vs AttAcc>

42.7×

vs CPU (M4)

HBM3 config

2.64×

vs RTX 3090

HBM3 config

1.24×

vs AttAcc (4 PIM)

HBM3 config

1.45×

vs AttAcc (8 PIM)

HBM3 config

Evaluation: Area, Power, Model accuracy

Area & Power

0.057 mm²

Softmax Unit Area

6.06%

% of HBM-PIM PU area

54.01%

Softmax Power vs FP16 Adder

Modest area and power overhead — fits within the PIM execution-unit power budget.

Model Accuracy (WikiText-103)

Method	PPL	Token Acc.
Softmax (baseline)	43.67	35.97%
SoftmaxPIM	46.72	35.04%

Key finding: SoftmaxPIM achieves near-identical accuracy to ConSmax (46.72 vs 46.70 PPL) while significantly reducing numerical approximation errors through higher-precision LUT-based computation.

Conclusion

- ◆ **SoftmaxPIM integrates a lightweight near-bank softmax unit into HBM-PIM to execute the full GeMV–Softmax pipeline entirely within the memory stack.**
- ◆ **SoftmaxPIM eliminates costly global reduction steps, enabling fully independent, parallel softmax execution without host memory transfers.**
- ◆ **Achieves up to 42.7× speedup over CPU, 2.64× over GPU, and 1.45× over the prior PIM accelerator AttAcc — with minimal area and power overhead.**
- ◆ **Maintains model accuracy close to exact softmax, with negligible perplexity degradation on WikiText-103.**

Conclusion

Conclusion

Toward Practical xPU–PIM Heterogeneous Systems

OUR FINAL GOAL

A unified xPU + PIM heterogeneous system



MINIMIZE DATA MOVEMENT

Keep computation close to data whenever possible

EXPAND KERNEL COVERAGE

Current PIM accelerates only selected kernels, so xPU–PIM cooperation is necessary.

PRESERVE SYSTEM COMPATIBILITY

Minimal hardware changes make PIM easier to integrate into real systems.

Thank you